



Step-wise Rubric Rewards for LLM Reasoning

**Weichu Xie^{1,2,*}, Haozhe Zhao^{1,*}, Wenpu Liu^{1,2,*}, Yongfu Zhu², Liang Chen¹,
Minghao Ye³, Zirong Chen⁴, Yuqi Xu^{1,2}, Shuai Dong^{5,2}, Ziyue Wang^{1,2}, Xinbo
Xu¹, Kean Shi¹, Ruoyu Wu¹, Xiaoying Zhang², Wenqi Shao², Baobao
Chang^{1,†}, Nan Duan², Jiaqi Wang²**

¹Peking University, ²JD Explore Academy, ³Shanghai Jiao Tong University, ⁴Tsinghua University, ⁵Shanghai Innovation Institute

*Equal contribution, †Corresponding author.

Abstract

Reinforcement Learning with Verifiable Rewards (RLVR) is widely used to improve the reasoning capabilities of large language models, but its reward is derived only from the correctness of the final answer and provides no supervision over intermediate reasoning steps. Recent rubric-based methods, such as Rubrics as Rewards (RaR), introduce finer-grained supervision by scoring roll-outs against a structured set of evaluation criteria. However, the resulting rubric scores are still aggregated into a single scalar that is applied to the entire response, leading to three structural weaknesses, namely loss of the multi-criterion rubric structure, uniform supervision of correct and incorrect reasoning steps, and reward hacking in the trained model through unbounded self-correction. On a sample of 1,000 problems, we find that 18.2% of steps within answer-correct responses are themselves wrong yet positively rewarded, while 49.9% of steps within answer-incorrect responses are in fact correct yet penalized. Therefore, we introduce **Step-wise Rubrics as Rewards (SRaR)**, an RLVR framework that (i) uses an LLM judge to attribute each rubric item to a specific reasoning step, (ii) normalizes the per-step rubric scores across rollouts so that only steps whose quality varies produce a learning signal, and (iii) combines the resulting per-step reward with the standard outcome reward through a decoupled advantage estimator that keeps the outcome-driven baseline stable. To support training, we further build a 16K-problem rubric dataset by contrastively distilling rubric items from correct and flawed reasoning paths sampled from a strong model and verified against the ground-truth answer. Across six mathematical reasoning benchmarks spanning multiple difficulty levels, SRaR improves the average accuracy over RaR by **3.57** points on Qwen3-8B-Non-Thinking and by **2.75** points on Qwen3-32B-Non-Thinking, raises the Faithful Reasoning Rate on AIME 2025 from 34.5% to 46.7% (every correct-answer trajectory of SRaR uses entirely correct reasoning steps), and reduces the rate of self-correction looping from 48.1% to 26.5%.

Date: May 7, 2026

Code: <https://github.com/>

Correspondence: chbb@pku.edu.cn

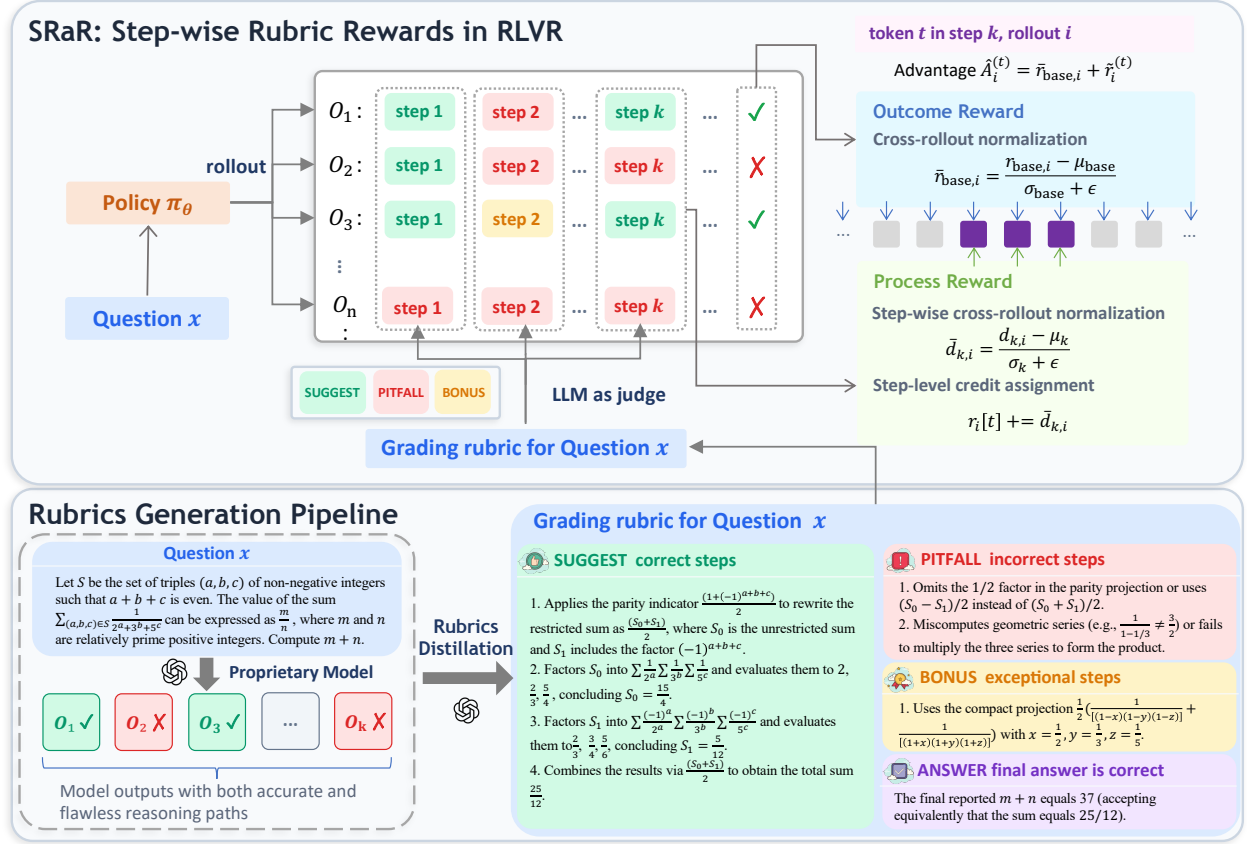


Figure 1 Overview of SRaR. Top: The SRaR training framework. An LLM judge attributes each rubric item to a specific reasoning step; per-step rewards are normalized across rollouts and combined with outcome rewards via a decoupled advantage estimator. **Bottom:** The three-stage contrastive rubric distillation pipeline that generates instance-specific rubrics from correct and flawed paths.

1 Introduction

Large language models (LLMs) have demonstrated remarkable reasoning capabilities across complex tasks such as mathematics, programming, and scientific problem solving [11, 16]. To further enhance these capabilities, Reinforcement Learning with Verifiable Rewards (RLVR) has emerged as a widely adopted training paradigm in which rule-based verifiers automatically assess the correctness of model-generated answers and produce reward signals for policy optimization [2, 12]. While effective, the RLVR reward is derived solely from the correctness of the final answer and provides no supervision over the quality of intermediate reasoning steps. To inject finer-grained supervision, recent work turns to rubrics, structured checklists of evaluation criteria that decompose response quality into multiple verifiable aspects and are scored automatically by an LLM judge [4, 7]. Methods such as Rubrics as Rewards (RaR) [3] and RGR-GRPO [1] use these rubric scores as RL reward signals by aggregating the per-criterion scores into a single scalar reward applied to the entire response (Figure 2(a)).

Despite providing richer supervision than pure outcome rewards, current rubric-based methods exhibit three structural weaknesses. **First, existing methods score each rubric item as binary pass/fail and then merge all per-step verdicts into a single response-level scalar, discarding both the type-level and step-level structure of the rubric.** The binary scoring erases the distinction between criteria that reinforce standard reasoning, flag common errors, and credit exceptional insights. The response-level aggregation further discards which step each criterion judges, leaving the policy gradient with no information about the multi-aspect structure that motivated using rubrics in the first place. **Second, this response-level reward is applied uniformly to every token, so correct and incorrect reasoning steps within the**

same response receive the same reward signal. As a result, the policy gradient cannot distinguish steps that actually contributed to the response’s quality from steps that should have been corrected. To measure how often this matters in practice, we sample 1,000 random solutions from Qwen3-8B-Non-Thinking on the DAPO training set and independently judge each reasoning step. 18.2% of steps within answer-correct responses are themselves wrong and 49.9% of steps within answer-incorrect responses are themselves correct (Figure 2(b)), so any single-scalar response-level reward assigns incorrect supervision to a substantial fraction of intermediate steps. **Third, the rubric score rewards visible reasoning effort such as self-correction and re-verification, so the model can inflate it without actually improving the answer.** Because existing methods feed this rubric score directly into the policy gradient, the model learns to over-produce such surface behaviors as a form of reward hacking. On AIME 2025, this manifests as severe self-correction looping: 48.1% of RaR rollouts repeatedly rewrite and re-verify intermediate steps to inflate the rubric reward, often failing to terminate (§ 4.3.2). **This raises a natural question: how can a reward signal be both interpretable and fine-grained in supervising a model’s reasoning?**

First, step-attributed rubric judging preserves the multi-aspect structure that existing aggregation discards. Each rubric item carries one of three types: SUGGEST for standard reasoning, PITFALL for known errors, and BONUS for exceptional insights. An LLM judge evaluates whether each item is satisfied and ties it to the reasoning step it judges, producing per-step, type-differentiated rewards that are retained at the step level rather than merged into a single response-level scalar. **Second, per-step token assignment with cross-rollout normalization** replaces the uniform token-level reward with step-aligned credit. Each step’s reward is broadcast only to its own tokens and normalized across rollouts of the same prompt, so each token receives a learning signal aligned with its own step. **Third, a decoupled advantage estimator** computes the advantage as a sum of two separately normalized parts: one from the outcome reward, one from the rubric. The outcome part is the standard advantage from accuracy alone; the rubric part is added on top as a bounded, per-step adjustment. Because rubric noise stays in the rubric part, no rollout can inflate its overall advantage by gaming the rubric, removing the incentive behind self-correction looping and yielding more stable training.

All three designs assume access to a problem-specific rubric, yet no public corpus provides rubrics paired with reasoning data at the scale needed for RL training. We therefore introduce a **contrastive rubric distillation** pipeline that constructs a 16K-problem rubric corpus in three stages: (i) sampling multiple reasoning rollouts from a strong model (GPT-5) per problem; (ii) verifying them against the ground-truth answer to split correct from flawed reasoning paths; and (iii) feeding both groups back to the same model to distill rubric items, each classified as SUGGEST, PITFALL, or BONUS, that capture what separates successful from failed reasoning. By grounding each rubric item in empirically observed failure modes rather than a single reference answer, this procedure yields criteria that target the errors a policy is actually likely to commit during RL training.

We evaluate *SRaR* on various reasoning benchmarks spanning multiple difficulty levels and model scales. *SRaR* improves average accuracy over the strongest baseline by **3.57** points on Qwen3-8B and **2.75** points on Qwen3-32B, with consistent improvements across all benchmarks and model scales. Three targeted analyses on AIME 2025 attribute these gains to the three designs above. Step-attributed judging with per-step token assignment raises the Faithful Reasoning Rate, the fraction of trajectories with both a correct answer and all-correct intermediate steps, from 34.5% to 46.7%. The resulting step-level credit assignment reduces self-correction looping, a failure mode in which the model repeatedly regenerates or revisits reasoning steps without converging, from 48.1% to 26.5%. Finally, removing cross-rollout step normalization causes training to collapse. These results demonstrate that step-level rubric supervision improves both final-answer accuracy and intermediate reasoning quality.

Our contributions are summarized as follows:

1. We identify three structural weaknesses of previous methods that use rubric scores as RL reward signals for finer-grained supervision and quantify their impact via a diagnostic study on 1,000 random model rollouts.
2. We propose *SRaR*, an RLVR framework that delivers step-level rubric supervision via three coordinated designs that provide interpretable and fine-grained reward signals for supervising model reasoning.

Query Let S be the set of triples (a, b, c) of non-negative integers with $a + b + c$ even. If $\sum_{(a,b,c) \in S} 2^{-a} 3^{-b} 5^{-c} = \frac{m}{n}$ in lowest terms, compute $m + n$.	
Response Step 1: Parity indicator : rewrites $S = \frac{1}{2}(S_0 + S_1)$. Step 2: GF projection : $S = \frac{1}{2} \left[\prod_{1-x_i} + \prod_{1+x_i} \right]$, $x_i \in \left\{ \frac{1}{2}, \frac{1}{3}, \frac{1}{5} \right\}$ Step 3: Evaluates $\prod_{1-x_i} = 2 \cdot \frac{3}{2} \cdot \frac{5}{4} = \frac{15}{4}$ Step 4: Evaluates $\prod_{1+x_i} = \left(\frac{2}{3} \right) \left(\frac{5}{6} \right) = \frac{5}{24}$ Step 5: $S = \frac{1}{2} \left(\frac{15}{4} + \frac{5}{24} \right) = \frac{95}{48}$, $m + n = 143$	
Trajectory-level Rubric Reward (RaR) ✓ SUGGEST Applies parity indicator; rewrites S as $S = \frac{1}{2}(S_0 + S_1)$. ✓ BONUS Uses compact GF projection $S = \frac{1}{2} \left[\prod_{1-x_i} + \prod_{1+x_i} \right]$. ✗ PITFALL Drops the $\frac{1}{2}$ factor: sums $S_0 + S_1$ unhalved, giving $S = \frac{25}{6}$, so $m+n = 31 \neq 37$. Overall Rubric Score: 0.2 One score for the entire trajectory.	
Step-Wise Rubric Reward (Ours) ✓ SUGGEST Applies parity indicator; rewrites S as $S = \frac{1}{2}(S_0 + S_1)$. Step 1 ✓ BONUS Uses compact GF projection $S = \frac{1}{2} \left[\prod_{1-x_i} + \prod_{1+x_i} \right]$. Step 2 ✗ PITFALL Drops the $\frac{1}{2}$ factor: sums $S_0 + S_1$ unhalved, giving $S = \frac{25}{6}$, so $m+n = 31 \neq 37$. Step 3 Step1 Rubric Score: 0.2 Step2 Rubric Score: 0.4 Step3 Rubric Score: -0.4 Per-step score—each rubric item attributes to its step.	

(a) Response-level (RaR) vs. step-wise (SRaR) rubric reward

Category	# Samples	# Steps	Reward	Indiscriminate
A✓S✓	194	1012	+	—
A✓S×	194	916	+	350 wrong
A×S✓	18	101	−	101 correct
A×S×	591	3151	−	1521 correct

Indiscriminate reward under response-level signal:

Wrong steps receiving same + as correct (A✓):

$$\frac{350}{1012 + 916} = \frac{350}{1928} = 18.2\%$$

Correct steps receiving same − as wrong (A×):

$$\frac{101 + 1521}{101 + 3151} = \frac{1622}{3252} = 49.9\%$$

(b) Indiscriminate reward under response-level signal

Figure 2 Motivation for step-wise rubric reward. (a) RaR applies a single scalar rubric reward to the entire response, so every step receives the same signal; SRaR decomposes the rubric into per-step rewards aligned with each reasoning step. (b) Diagnostic study on 1,000 random Qwen3-8B-Non-Thinking solutions from the DAPO training set, each parsed into reasoning steps and independently judged (details in § 4.3.1). A✓/A× denotes correct/incorrect final answer; S✓/S× denotes all steps correct / at least one step wrong. Under a single-scalar response-level reward, **18.2%** of steps within answer-correct responses are wrong yet receive the same + reward as correct steps, and **49.9%** of steps within answer-incorrect responses are correct yet receive the same − penalty as wrong steps.

3. We release a 16K-problem rubric dataset built via correctness-verified contrastive distillation. *SRaR* outperforms RL baselines by a large margin on six reasoning benchmarks across different model scales, while raising the Faithful Reasoning Rate and substantially reducing the self-correction looping observed in other methods.

2 Related Work

2.1 Process Supervision for Reasoning

RLVR with GRPO and DAPO [19] provides only sparse binary outcome signals with no guidance on intermediate steps. Process Reward Models (PRMs) [9, 10, 14] score individual steps but require massive human annotations and produce opaque scalars with no explanation for why a step receives its score. GRPO-VPS [13] avoids auxiliary models by measuring the policy’s own conditional probability of the correct answer at each step boundary, but the resulting signal is still a single opaque scalar per step that offers no interpretable diagnosis of reasoning quality.

2.2 Rubric-Based Evaluation and Rewards

Rubric-based evaluation decomposes quality into interpretable criteria without auxiliary models. Hashemi et al. [4] introduce LLM-Rubric for calibrated multi-aspect evaluation; Kim et al. [7] train open-source rubric-guided evaluators; RaR [3] and RGR-GRPO [1] use rubric scores directly as RL rewards. However, all existing rubric-based RL methods aggregate scores into a single trajectory-level scalar, forfeiting per-step

credit: correct steps in failed trajectories are penalized and incorrect steps in successful ones are reinforced. SRaR addresses this overlooked problem by attributing each rubric item to its corresponding reasoning step and normalizing rewards at the step level.

3 Method

3.1 Preliminary

RLVR for LLM Reasoning. Reinforcement Learning with Verifiable Rewards (RLVR) trains a policy π_θ by rewarding responses whose final answer matches the ground truth [2, 12]. We adopt Group Relative Policy Optimization (GRPO) [12]: for each prompt x , GRPO samples n rollouts $\{y_i\}_{i=1}^n \sim \pi_\theta(\cdot | x)$, scores each with a scalar reward r_i , and computes the advantage by group normalization

$$A_i = \frac{r_i - \mu}{\sigma + \epsilon}, \quad \mu = \frac{1}{n} \sum_{j=1}^n r_j, \quad \sigma^2 = \frac{1}{n} \sum_{j=1}^n (r_j - \mu)^2. \quad (1)$$

The same scalar A_i is broadcast as the per-token advantage at every position t of y_i .

Problem Formulation. Each problem x in our training set $\mathcal{D} = \{(x, a^*, \mathcal{R}_x)\}$ carries a ground-truth answer a^* and a rubric $\mathcal{R}_x = \{(c_j, \tau_j)\}_{j=1}^M$ with criterion text c_j and type $\tau_j \in \{\text{SUGGEST}, \text{PITFALL}, \text{BONUS}\}$. Given a response y with T tokens organized into K steps with token spans $\{(a_k, b_k)\}_{k=1}^K$, our task is to construct a token-level advantage $\hat{A}_i^{(t)}$ for each rollout i and position t , jointly informed by outcome correctness and rubric verdicts, as a drop-in replacement for A_i .

3.2 Rubrics Design

A rubric $\mathcal{R}_x$ comprises three item types, each contributing a distinct sign to the policy gradient (Section 3.3). **SUGGEST** items reinforce standard problem-solving paradigms (clear step structure, logical coherence, explicit derivations), contributing positive deltas. **PITFALL** items penalize known failure modes (arithmetic errors, logical leaps, hallucinations), contributing negative deltas. **BONUS** items reward valuable optional behaviors (alternative solution paths, cross-validation), contributing additional positive deltas. This three-aspect schema separates baseline correctness, error avoidance, and exceptional quality, each bounded by a per-type budget.

3.3 Step-wise Rubrics Rewards in RLVR (SRaR)

Figure 1 (top) provides an overview. SRaR realizes three coordinated designs: **step-attributed rubric judging**, **per-step token assignment with cross-rollout normalization**, and a **decoupled advantage estimator**.

Step-attributed rubric judging. An LLM judge ties each rubric item to the reasoning step it judges, producing a per-step, per-criterion verdict instead of a single response-level scalar. To enable step attribution, the policy is prompted to emit responses in a structured step format (“### Step N: ...”) with the final answer in `\boxed{\}`, and step token spans $\{(a_k, b_k)\}$ are recovered via the tokenizer’s offset mapping. For each response y , the judge $\mathcal{J}$ [20] returns

$$\mathcal{J}(y, \mathcal{R}_x) = \{(s_j, k_j)\}_{j=1}^M, \quad s_j \in \{0, 1\}, \quad k_j \in \{1, \dots, K\}, \quad (2)$$

where s_j indicates whether criterion j is satisfied and k_j records the step it judges.

Per-step token assignment with cross-rollout normalization. Each judged rubric item is converted into a per-step reward, normalized across rollouts at the step level, and broadcast only to the tokens of the step it judges. This replaces the uniform token-level reward of standard GRPO with step-aligned credit.

We first map each verdict to a signed delta whose magnitude depends on its rubric type:

$$\delta_j = \begin{cases} +R_{\text{SUG}}/N_{\text{SUG}} & \tau_j = \text{SUGGEST}, s_j = 1 \\ -|R_{\text{PIT}}|/N_{\text{PIT}} & \tau_j = \text{PITFALL}, s_j = 1 \\ +R_{\text{BON}}/N_{\text{BON}} & \tau_j = \text{BONUS}, s_j = 1 \\ 0 & \text{otherwise,} \end{cases} \quad (3)$$

where R_τ is the total reward budget for type τ and N_τ counts the items of that type in the current sample, keeping each category’s contribution bounded regardless of rubric list length. Deltas attributed to the same step k are summed into a per-step raw rubric reward $d_{k,i} = \sum_{j:k_j=k} \delta_j$ for rollout i , representing the total rubric credit that step k receives before normalization.

To extract a learning signal that reflects relative step quality across rollouts, we normalize $d_{k,i}$ per step across rollouts of the same prompt:

$$\bar{d}_{k,i} = \frac{d_{k,i} - \mu_k}{\sigma_k + \epsilon}, \quad \mu_k = \frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} d_{k,i}, \quad (4)$$

where $\mathcal{G}_k$ is the set of rollouts whose judge evaluation contains at least one rubric item attributed to step k , and $\bar{d}_{k,i} = 0$ when $|\mathcal{G}_k| \leq 1$ since a single sample provides no relative signal. This mirrors GRPO’s group normalization, applied independently per step: steps that are consistently good or bad across rollouts receive near-zero advantage, while steps whose quality varies receive a strong learning signal. The normalized $\bar{d}_{k,i}$ is then broadcast to every token within the step’s span:

$$\tilde{r}_i^{(t)} = \bar{d}_{k,i} \quad \text{for all } t \in [a_k, b_k], \quad (5)$$

so that all tokens of a reasoning step share the same rubric-derived signal.

Decoupled advantage estimator. A naive substitution of the combined base-and-rubric reward into the GRPO advantage would let rubric noise enter the group baseline, allowing a rollout to inflate its overall advantage by gaming the rubric. We instead compute the advantage as a sum of two separately normalized parts:

$$\hat{A}_i^{(t)} = \underbrace{\frac{r_{\text{base},i} - \mu_{\text{base}}}{\sigma_{\text{base}} + \epsilon}}_{\text{outcome advantage}} + \underbrace{\tilde{r}_i^{(t)}}_{\text{rubric offset}}, \quad (6)$$

where $r_{\text{base},i} = (1 - \lambda)r_{\text{acc},i} + \lambda r_{\text{fmt},i}$ combines an accuracy reward $r_{\text{acc},i} \in \{0,1\}$ for whether the extracted answer is correct and a format reward $r_{\text{fmt},i} \in \{0,1\}$ for whether the response contains both step headers and a `\boxed{\}` answer, and $\mu_{\text{base}}, \sigma_{\text{base}}$ are computed across rollouts of the same prompt. The outcome term is the standard GRPO advantage from accuracy alone; the rubric term is the bounded, per-step offset constructed above. Because rubric noise stays in the offset and never enters the group baseline, no rollout can inflate its overall advantage by gaming the rubric, removing the incentive behind self-correction looping and yielding more stable training. We provide formal analyses of the estimator’s zero-sum, bounded, and unbiased properties in Appendix O.

Algorithm 1 SRaR: Step-wise Rubric Rewards in RLVR

Require: Policy π_θ , training set $\mathcal{D}$ with rubrics, LLM judge $\mathcal{J}$, group size n

```
1: for each training iteration do
2:   Sample prompts  $\{x\}$  from  $\mathcal{D}$ ; for each  $x$ , sample  $n$  rollouts  $\{y_i\}_{i=1}^n \sim \pi_\theta(\cdot | x)$ 
3:   for each prompt  $x$  do
4:     for  $i = 1, \dots, n$  do
5:        $r_{\text{base},i} \leftarrow (1 - \lambda) r_{\text{acc},i} + \lambda r_{\text{fmt},i}$ 
6:       Detect step spans  $\{(a_k^i, b_k^i)\}_{k=1}^{K_i}$  via tokenizer offset mapping
7:        $\{(s_j, k_j)\}_j \leftarrow \mathcal{J}(y_i, \mathcal{R}_x)$  ▷ Step-attributed rubric judging
8:        $d_{k,i} \leftarrow \sum_{j: k_j=k} \delta_j$  for each step  $k$ 
9:     end for
10:    for each step  $k$  present in any rollout do
11:       $\mathcal{G}_k \leftarrow \{i : d_{k,i} \text{ exists}\}; \bar{d}_{k,i} \leftarrow 0$  if  $|\mathcal{G}_k| \leq 1$  else  $(d_{k,i} - \mu_k)/(\sigma_k + \epsilon)$ 
12:    end for
13:    for  $i = 1, \dots, n$  do
14:       $\tilde{r}_i^{(t)} \leftarrow \bar{d}_{k,i}$  for all  $t \in [a_k^i, b_k^i]$ , each step  $k$  ▷ Per-step token assignment
15:       $A_{\text{base},i} \leftarrow (r_{\text{base},i} - \mu_{\text{base}})/(\sigma_{\text{base}} + \epsilon)$ 
16:       $\hat{A}_i^{(t)} \leftarrow A_{\text{base},i} + \tilde{r}_i^{(t)}$  for all  $t$  ▷ Decoupled advantage
17:    end for
18:  end for
19:  Update  $\theta$  via clipped PPO with token-level advantages  $\{\hat{A}_i^{(t)}\}$ 
20: end for
```

3.4 Contrastive Rubric Distillation

No public corpus provides problem-specific rubrics paired with reasoning data at scale. We construct a 16K-problem rubric corpus via contrastive distillation (Figure 1, bottom) in three stages. **(1) Reasoning rollout generation.** We prompt GPT-5 to sample multiple step-by-step trajectories per problem with final answers in a standardized tag. **(2) Trajectory verification.** Each trajectory is matched against the ground-truth answer, partitioning rollouts into correct and flawed paths. **(3) Contrastive rubric distillation.** Both correct and flawed paths are fed back to the model, which contrasts structural differences to distill a fine-grained rubric list. Grounding items in empirically observed failures yields criteria targeting errors a policy is likely to commit during training.

Prompts are in Appendix B.1 and B.2; rubric examples in Appendix I; quality evaluation in Appendix F; judge validation in Appendix G.

4 Experiment

4.1 Experimental Setup

Training Details. We build the training set by applying our pipeline to the DAPO math dataset (statistics in Appendix P) and adopt the same hyperparameters and training setting as DAPO except for the reward signal. Policy models are Qwen3-8B-Non-Thinking and Qwen3-32B-Non-Thinking [18], trained on 8× NVIDIA H200 GPUs with rubric budgets $R_{\text{SUG}} = 0.8$, $R_{\text{PIT}} = -1.0$, $R_{\text{BON}} = 1.0$ and format weight $\lambda = 0.1$. Details are in Appendix C and B.3.

Evaluation Configuration. We evaluate on six benchmarks: AIME 2024/2025, AMC 2023, MATH500 [6], Minerva Math [8], and Olympiad Bench [5]. Decoding uses temperature 0.7, top- p 0.8, presence penalty 1.5, repetition penalty 1.0, max length 10,240 tokens. AIME 2024/2025 and AMC 2023 report Avg@32 [15]; remaining benchmarks use a single generation.

Baselines. We compare against GRPO [12], DAPO [19], RaR [3], GRPO-VPS [13], and RGR-GRPO [1]. The latter two are re-implemented under the same training recipe for a fair comparison.

Table 1 Main results on six mathematical reasoning benchmarks for Qwen3-8B-Non-Thinking and Qwen3-32B-Non-Thinking. Δ denotes the improvement of SRaR over the strongest existing baseline (RaR). The best results are **bolded**, second best are underlined, and our method is highlighted in light red. For AIME 2024/2025 and AMC 2023, we report Avg@32.

Method	AIME24	AIME25	AMC23	MATH500	Minerva	Olympiad	Avg.
<i>Qwen3-8B-Non-Thinking</i>							
Baseline	26.88	23.33	72.03	86.8	55.15	56.23	53.40
+ GRPO	36.56	27.71	79.06	89.2	55.51	63.20	58.54
+ GRPO-VPS	44.58	32.40	83.20	88.60	<u>61.03</u>	60.24	61.68
+ RGR-GRPO	45.31	34.69	81.09	90.40	<u>61.03</u>	66.91	63.24
+ DAPO	52.60	41.35	<u>90.23</u>	92.0	59.56	<u>67.80</u>	67.26
+ RaR	<u>57.50</u>	<u>43.23</u>	88.20	<u>92.6</u>	64.71	67.51	<u>68.96</u>
+ SRaR (Ours)	61.15	49.06	92.73	94.4	64.71	73.15	72.53
Δ	+3.65	+5.83	+4.53	+1.80	+0.00	+5.64	+3.57
<i>Qwen3-32B-Non-Thinking</i>							
Baseline	27.19	21.67	71.48	85.8	62.87	58.75	54.63
+ GRPO	37.92	30.21	80.78	91.0	63.97	66.02	61.65
+ GRPO-VPS	43.02	26.56	80.08	90.20	63.97	64.54	61.40
+ RGR-GRPO	47.71	36.46	87.42	94.40	<u>68.01</u>	<u>72.40</u>	67.73
+ DAPO	61.25	47.81	93.67	95.4	63.60	72.11	72.31
+ RaR	<u>63.65</u>	<u>50.94</u>	<u>94.22</u>	<u>94.8</u>	65.44	69.29	<u>73.06</u>
+ SRaR (Ours)	64.17	51.77	94.77	95.4	72.06	76.71	75.81
Δ	+0.52	+0.83	+0.55	+0.60	+6.62	+7.42	+2.75

Rubric LLM Judge. We use GPT-OSS-20B as the rubric judge (prompt in Appendix B.4). Robustness to judge choice is validated with Qwen3-30B-A3B-Instruct as an alternative (Appendix H).

4.2 Experimental Results

4.2.1 Main Results

Table 1 reports the main results. SRaR achieves the highest average accuracy at both scales: 72.53 on Qwen3-8B (+3.57 over RaR) and 75.81 on Qwen3-32B (+2.75 over RaR), and improves over GRPO-VPS by 10.85/14.41 points and over RGR-GRPO by 9.29/8.08 points. The advantage over GRPO-VPS stems from rubric items describing specific reasoning behaviors rather than relying on probability increments of a reference answer, enabling credit attribution to step content rather than statistical effects. The advantage over RaR and RGR-GRPO comes from routing each rubric item to the step it evaluates, eliminating indiscriminate supervision where correct steps in incorrect trajectories are penalized. We validate this in § 4.3.1.

The gains concentrate on longer benchmarks: at 8B, AIME 2025 (+5.83), Olympiad Bench (+5.64), and AMC 2023 (+4.53) improve most, while MATH500 (+1.80) and Minerva move less. Across scales, SRaR raises the base model’s average by 19.13 points at 8B and 21.18 points at 32B. Training dynamics are in Appendix D.

4.2.2 Think Longer and Better

Why gains vary across benchmarks. We correlate the average number of reasoning steps per benchmark with SRaR’s improvement over RaR (Figure 3, left). The correlation is strong (Pearson $r = 0.91$, $p = 0.012$): benchmarks requiring more steps (AIME 2025: 3.6, Olympiad: 3.0) benefit substantially more than those with shorter solutions (Minerva: 1.6).

Harder problems benefit more. We stratify Olympiad Bench problems into difficulty quintiles by base

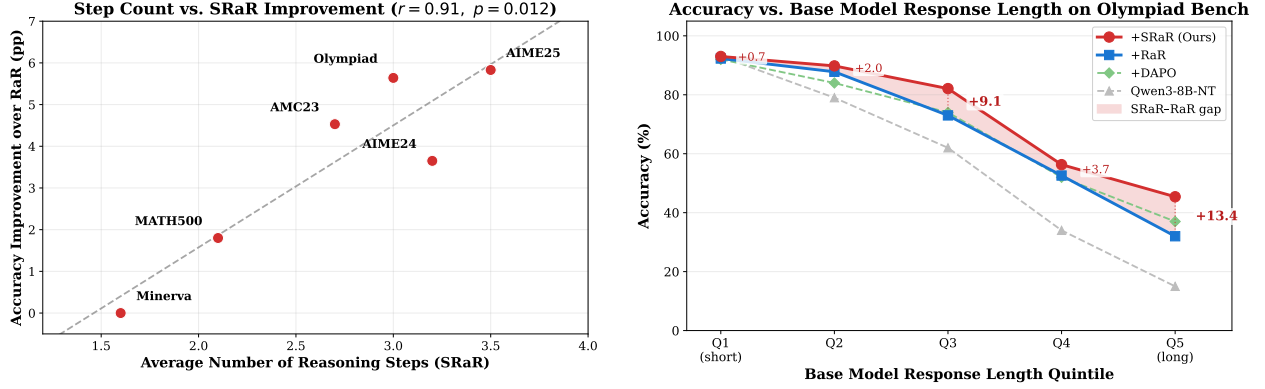


Figure 3 (Left) Correlation between the average number of reasoning steps per benchmark and SRaR’s accuracy improvement over RaR (Pearson $r = 0.91, p = 0.012$). Benchmarks requiring more reasoning steps benefit more from step-wise credit assignment. **(Right)** Accuracy vs. problem difficulty on Olympiad Bench (674 problems), where difficulty quintiles are defined by the base model’s response length. SRaR outperforms all baselines across all five quintiles, with the largest margin on the hardest problems (Q5: +13.4 pp over RaR).

model response length (Figure 3, right). SRaR outperforms RaR in all quintiles, with the gap widening from +0.7 pp (Q1) to +13.4 pp (Q5). On Q5, SRaR also generates shorter responses (3,645 vs. 5,933 tokens), confirming gains from effective reasoning rather than exhaustive self-revision.

4.3 Analysis

4.3.1 Process-Outcome Alignment of Step-wise Rewards

To examine whether step-level reward assignment improves intermediate reasoning quality beyond final-answer accuracy, we conduct a diagnostic on AIME 2025 (single-pass). For each model, we parse solutions into steps and an independent LLM judge evaluates each step’s correctness conditioned on preceding steps (prompt in Appendix B.5).

As shown in Table 2, the base model reaches only 60.8% step accuracy, confirming that outcome-only supervision fails to prevent error accumulation. RaR raises step accuracy to 80.8% but still has 2 correct-answer trajectories with erroneous steps ($A\sqrt{S}\times=2$) and 4 incorrect-answer trajectories with all-correct steps ($A\times S\sqrt{=4}$). SRaR achieves the highest step accuracy (85.2%) and answer accuracy (46.7%), and is the only model with $A\sqrt{S}\times=0$: every correct-answer trajectory uses entirely correct reasoning. SRaR also attains the highest Faithful Reasoning Rate (46.7% vs. 34.5%) and the lowest Misaligned Trajectory Rate (10.0% vs. 20.7%), showing that step-level rewards align intermediate quality with outcome correctness. Case studies are in Appendix J.1.

4.3.2 Reduction of Repetitive Looping

A common failure mode of RLVR-trained models is repetitive looping: unbounded self-correction cycles that revisit computations without converging (case study in Appendix J.2). We quantify this on 960 AIME 2025 samples (30 problems, 32 rollouts each), classifying a response as looping if any of four criteria holds: (i) > 20 self-correction phrases; (ii) `### Step 1` appearing more than once; (iii) repeated step headings; or (iv) $> 10\%$ duplicate paragraphs.

As shown in Table 3, RaR’s looping rate is 48.1% with a mean of 25.4 self-correction phrases, and its looping samples achieve only 24.2% accuracy vs. 53.4% for non-looping ones. SRaR reduces the looping rate to 26.5% with mean self-correction count 14.8, and its looping samples reach 42.5% accuracy, suggesting productive reflection rather than uncontrolled repetition.

We attribute this to step-level credit assignment: each step receives an independent reward, so the model can commit to completed steps rather than perpetually revisiting them.

Table 2 Step-level reasoning quality analysis on AIME 2025. **Step Acc.** is the percentage of steps judged correct. The four combination columns show counts per (answer $\times$ step correctness) category. **FRR** (Faithful Reasoning Rate) is $A\check{S}\check{S} / \text{total} (\uparrow)$, the fraction of trajectories with both a correct answer and all-correct steps. **MTR** (Misaligned Trajectory Rate) is $(A\check{S}\times + A\times S\check{S}) / \text{total} (\downarrow)$, the fraction of trajectories whose step-level judgement disagrees with the outcome, and serves as the population-level analogue of the misalignment in Figure 2(b).

Model	Ans.	Step	Answer $\times$ Step				FRR	MTR
	Acc.	Acc.	$A\check{S}\check{S}$	$A\check{S}\times$	$A\times S\check{S}$	$A\times S\times$	(%) $\uparrow$	(%) $\downarrow$
Qwen3-8B	13.3	60.8	4	0	4	20	13.3	13.3
+ RaR	41.4	80.8	<u>10</u>	2	<u>4</u>	<u>12</u>	<u>34.5</u>	20.7
+ SRaR (Ours)	46.7	85.2	14	0	3	12	46.7	10.0

Table 3 Repetitive looping analysis on AIME 2025. **Loop Rate** is the percentage of samples exhibiting any form of looping. Accuracy is reported for looping vs. non-looping responses. **Mean S-C** denotes the average count of self-correction phrases per response.

Model	Loop Rate (%)	Acc. (loop)	Acc. (no loop)	Mean S-C
Qwen3-8B	56.8	25.9	16.1	0.9
+ RaR	48.1	<u>24.2</u>	53.4	25.4
+ SRaR (Ours)	26.5	42.5	<u>52.8</u>	<u>14.8</u>

4.4 Ablation Study

Rubric aspect ablation. We test whether PITFALL and BONUS provide value beyond Suggest-only. As shown in Table 9, SRaR outperforms RaR under both configurations (+2.86 with Suggest-only, +3.57 with multi-aspect), confirming the primary gain comes from step-wise credit assignment. Adding PITFALL and BONUS further raises accuracy from 71.39 to 72.53, indicating complementary supervision beyond positive guidance alone.

Method design ablation. We ablate three components on Qwen3-8B (Table 10 in Appendix M). Removing the step-structured format prompt yields 67.85 average accuracy, matching standard DAPO (67.26), since step boundaries are required for attribution. Removing cross-rollout step normalization causes collapse to 56.76: without normalization, group-invariant rubric signals dominate the gradient, and the model degenerates to a single step (Appendix N). Replacing the decoupled advantage with a merged RaR-style scalar yields 72.06, slightly below full SRaR (72.53), because merging injects rubric variance into the GRPO baseline statistics.

5 Conclusion

We present *Step-wise Rubrics as Rewards (SRaR)*, an RLVR framework that delivers fine-grained supervision during RL training via step-level rubrics. Through *step-attributed rubric judging* and *cross-rollout step normalization*, each reasoning step receives a step-wise rubric reward, while a *decoupled advantage estimator* keeps RL training stable against rubric noise. On six mathematical benchmarks, *SRaR* outperforms the state-of-the-art by **3.57** points on Qwen3-8B and **2.75** points on Qwen3-32B, raises the Faithful Reasoning Rate from 34.5% to 46.7%, and reduces self-correction looping from 48.1% to 26.5%. We hope *SRaR* provides a practical paradigm for delivering fine-grained rubric supervision to RL-trained reasoners, showing that the granularity at which supervision is applied matters as much as its content. We discuss limitations and future directions in Appendix A.

References

- [1] Baolong Bi, Shenghua Liu, Yiwei Wang, Siqian Tong, Lingrui Mei, Yuyao Ge, Yilong Xu, Jiafeng Guo, and Xueqi Cheng. Reward and guidance through rubrics: Promoting exploration to improve multi-domain reasoning. arXiv preprint arXiv:2511.12344, 2025.
- [2] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. Nature, 645: 633–638, 2025.
- [3] Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. Rubrics as rewards: Reinforcement learning beyond verifiable domains. arXiv preprint arXiv:2507.17746, 2025.
- [4] Helia Hashemi, Jason Eisner, Corby Rosset, Benjamin Van Durme, and Chris Kedzie. LLM-Rubric: A multidimensional, calibrated approach to automated evaluation of natural language texts. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 13806–13834, 2024.
- [5] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 2024.
- [6] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. Advances in Neural Information Processing Systems, 34:7294–7305, 2021.
- [7] Seungone Kim, Juyoung Suk, Shayne Longpre, Bill Yuchen Lin, Jamin Shin, Sean Welleck, Graham Neubig, Moontae Lee, Kyungjae Lee, and Minjoon Seo. Prometheus 2: An open source language model specialized in evaluating other language models. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024.
- [8] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. Advances in Neural Information Processing Systems, 35:3843–3857, 2022.
- [9] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In International Conference on Learning Representations, 2024.
- [10] Liangchen Luo, Yinxiao Liu, Rosanne Liu, Samrat Phatale, Meiqi Guo, Harsh Lara, Yunxuan Li, Lei Shu, Yun Zhu, Lei Meng, Jiao Sun, and Abhinav Rastogi. Improve mathematical reasoning in language models by automated process supervision. arXiv preprint arXiv:2406.06592, 2024.
- [11] OpenAI. Learning to reason with LLMs. <https://openai.com/index/learning-to-reason-with-llms/>, 2024.
- [12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.
- [13] Jingyi Wang, Lei Zhu, Tengjin Weng, Song-Li Wu, Haochen Tan, Jierun Chen, Chaofan Tao, Haoli Bai, Lu Hou, Lifeng Shang, and Xiao-Ping Zhang. GRPO-VPS: Enhancing group relative policy optimization with verifiable process supervision for effective reasoning. arXiv preprint arXiv:2604.20659, 2026.
- [14] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. arXiv preprint arXiv:2312.08935, 2024.
- [15] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In International Conference on Learning Representations, 2023.
- [16] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In Advances in Neural Information Processing Systems, 2022.

- [17] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [18] An Yang, Anfeng Yang, Baosong Yang, Beichen Bi, Binyuan Hui, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [19] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [20] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2023.

A Limitations

We identify the following limitations of our work:

1. **Dependence on the LLM judge.** SRaR relies on an external LLM judge to evaluate rubric satisfaction and attribute each item to a specific reasoning step. Errors in step attribution or satisfaction judgments propagate into the reward signal. Although step-wise cross-rollout normalization mitigates the impact of systematic biases, judge inaccuracies on individual samples may still introduce noise into training.
2. **Additional inference cost.** Each training iteration requires invoking the LLM judge on every rollout, increasing wall-clock time and computational cost compared to standard GRPO or trajectory-level rubric methods. This cost scales linearly with the number of rollouts and rubric items per prompt.
3. **Domain scope.** Our experiments are restricted to mathematical reasoning. While the framework is domain-agnostic in principle, we have not verified its effectiveness on other multi-step reasoning tasks such as code generation, scientific reasoning, or logical deduction. The rubric generation pipeline may require domain-specific adaptation.
4. **Rubric generation relies on a strong model.** The three-stage data pipeline uses a frontier model (GPT-5) to generate rollouts and distill rubrics. The quality of the resulting rubrics is bounded by the capabilities of this model, and the pipeline’s applicability to settings where only weaker models are available remains unexplored.
5. **Step boundary detection.** SRaR assumes that the policy model produces responses with explicit step headers (### Step N). This requires a format prompt during training and may not generalize to free-form reasoning without structural delimiters.

B Prompt Details

This section provides the full prompts used in our data pipeline and training procedure.

B.1 Rollout Generation Prompt

Prompt for Rollout Generation

Solve the following math problem step by step. Please strictly follow these formatting instructions:

1. **Step Format:** Organize your reasoning into clear steps. Start each step with the header “### Step N:” (e.g., ### Step 1:, ### Step 2:).
2. **Final Answer:** The last line of your response must be enclosed in tags like this: “<answer>Answer</answer>”, where Answer is the calculated result.

Problem: {Problem}

B.2 Rubric Distillation Prompt

Rubric Distillation Prompt Template

You are designing a grading rubric for math problem solutions.

The original problem statement is: {problem_text}

You are given n model solutions with a correctness label. For each numbered solution: {solutions_block}

The true correct answer for this problem is: <ANSWER>{gold_answer}</ANSWER>

Based on the samples and the true answer, create a concise, verifiable rubric. Output ONLY structured items using the tags below (one item per line):

- <SUGGEST>: Mandatory. One scoring checkpoint per line. Each checkpoint checks whether the candidate CoT covers a key reasoning step from the standard solution path. Make it specific and objectively checkable from the text.
- <PITFALL>: Optional. Only include if you are confident. Flags reasoning traps or common errors to avoid.
- <BONUS>: Optional. Only include if you are confident. Rewards especially clever, efficient, or elegant reasoning patterns.
- <ANSWER>: Mandatory. One line that checks whether the final answer matches the ground truth above (allow equivalent numeric forms).

Guidelines:

- Select only the most essential scoring points; total lines ≤ 8 (SUGGEST/ANSWER mandatory; PITFALL/BONUS only if sure). Do NOT pad to the limit; stop once the main reasoning chain is covered.
- Each <SUGGEST> is exactly one scoring point and must be independently verifiable by an LLM judge from the candidate response.
- Do NOT include formatting or output-structure checks (e.g., headers, tags, markdown). Focus solely on mathematical / logical content steps and correctness of the final answer.
- Do not reference the given samples explicitly; generalize the rubric.
- Keep wording concise and actionable; no extra commentary before or after.

B.3 Training Format Prompt

Format Prompt

Solve the following math problem step by step. Follow these formatting rules:

1. **Steps:** Break your solution into multiple clear steps. Begin each step with “### Step N:” (e.g., ### Step 1:, ### Step 2:, ### Step 3:).
2. **Final Answer:** End your response with the answer inside \boxed{ }.

Problem: {Problem}

B.4 LLM Judge Prompt

LLM Judge Prompt

Problem

{problem}

Rubric Items

{rubric_items}

Student Solution

{response}

Student's Final Answer

{extracted_answer}

Task

For every rubric item listed above, evaluate the student's solution.

Rules:

- SUGGEST → satisfied: true if the student correctly performed that reasoning step.
- PITFALL → satisfied: true if the student made that mistake (fell into the pitfall).
- BONUS → satisfied: true if the student used that approach.

For step: the 1-indexed step number (### Step N) most closely associated with this item. Use 0 if it spans the whole solution; use -1 if there is no relevant step.

Return ONLY a valid JSON array:

```
[  
  {"id": <int>, "satisfied": <bool>, "step": <int>},  
  ...  
]
```

B.5 Step-Level Judge Prompt

Step-Level Correctness Judge Prompt

You are a math reasoning step verifier. Given a math problem and a sequence of reasoning steps, determine whether the CURRENT STEP is logically correct.

Problem

{question}

Ground Truth Answer

{answer}

Previous Steps (assumed correct for this evaluation)

{previous_steps}

Current Step to Evaluate

{current_step_title}

{current_step_content}

Instructions

Analyze whether the current step contains correct reasoning, calculations, and logic. Consider:

1. Are the mathematical operations correct?
2. Is the logic/deduction valid?
3. Are there any errors in the step?

Respond with EXACTLY one of:

- “CORRECT” if the step is logically and mathematically correct
- “INCORRECT” if the step contains any error

C Training Hyperparameters

Table 4 Training Hyperparameters.

Hyperparameter	Value
Max Prompt Length	2048
Max Response Length	8192
Train Batch Size	128
Learning Rate	1×10^{-6}
Rollout Group Size (n)	8
Training Steps	200
Temperature	1.0

D Training Dynamics

Figure 4 illustrates the training dynamics of different methods on Qwen3-32B-Non-Thinking. As shown in Figure 4(a), all three methods converge to similar training reward accuracy (~ 0.80 – 0.83), with SRaR reaching marginally higher values, indicating that step-level credit assignment provides effective gradient signals even at larger scale. Figure 4(b) reports validation accuracy. SRaR consistently achieves the highest validation accuracy throughout training, reaching ~ 0.47 at convergence compared to RaR (~ 0.43) and DAPO (~ 0.42), confirming that the training gains translate to held-out performance. Meanwhile, Figure 4(c) shows that both RaR and SRaR produce longer responses than DAPO, with similar length trajectories. Despite comparable response lengths, SRaR achieves substantially higher accuracy (Figure 4(b)), indicating that its gains stem

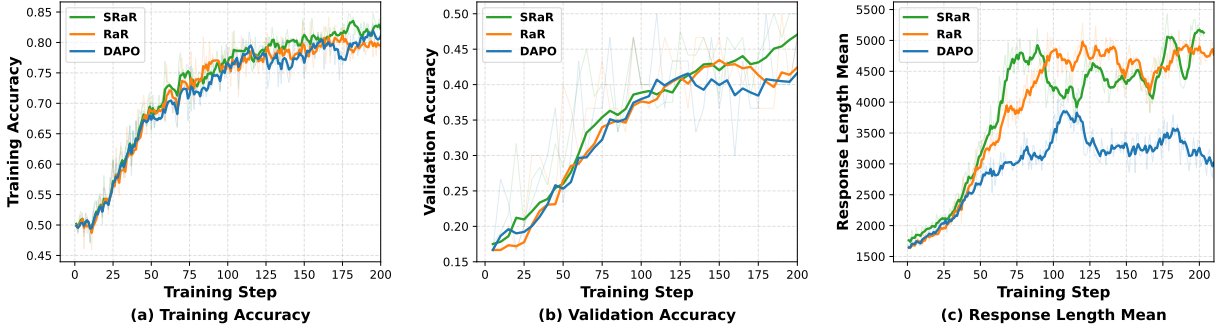


Figure 4 Training dynamics of DAPO, RaR, and SRaR on Qwen3-32B-Non-Thinking. **(a)** Training reward accuracy. **(b)** Validation accuracy. **(c)** Mean response length over training.

from more effective reasoning per token rather than simply generating more content. The per-aspect rubric score dynamics (Suggest, Bonus, and Pitfall) are provided in Figure 6.

E Format Compliance During Training

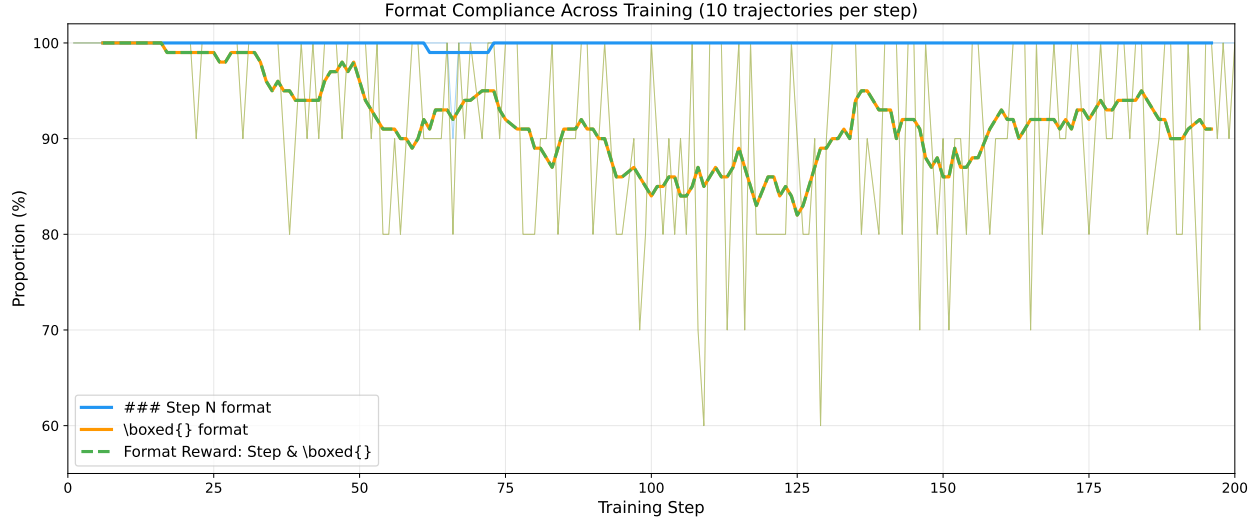


Figure 5 Format compliance across training steps for Qwen3-8B-Non-Thinking. At each training step, 10 rollouts are sampled and checked for (1) `### Step N` headers, (2) `\boxed{}` answer, and (3) the conjunction of both (i.e., the format reward). The step-header format is maintained at near-100% throughout training. The occasional drop in `\boxed{}` compliance is attributable to response truncation at the maximum generation length, which prevents the model from emitting a final boxed answer.

F Rubric Quality Evaluation

To assess the quality of rubrics produced by our pipeline, we conducted an automated validity evaluation on 51,583 rubric items spanning ~8,000 problems, using Gemini-3-Flash as the LLM judge. Each item was independently judged as *valid* or *invalid* with a brief justification. **Overall validity.** After this treatment, **98.0%** of evaluated rubric items are judged valid, with only 1,027 items (2.0%) flagged as invalid. **Validity by rubric type.** Table 5 shows validity rates broken down by rubric aspect. All three types achieve high and consistent validity rates in the range 96.3%–98.5%, confirming that the pipeline generates reliable rubrics across all aspects.

Table 5 Rubric validity rates by type, evaluated over 51,583 items from ~8,000 problems.

Rubric Type	Total	Valid	Validity Rate
SUGGEST	36,532	35,994	98.5%
PITFALL	9,822	9,458	96.3%
BONUS	5,213	5,089	97.6%
Overall	51,583	50,556	98.0%

Human--AI agreement. Two of the authors independently annotated a 100-item cross-check sample using the same prompt below, achieving 92% agreement with the LLM judge.

Rubric Validity Judge Prompt

You are a mathematics expert evaluating the quality of a rubric item for a math problem.

Math Problem

{problem}

Ground Truth Answer

{ground_truth}

Rubric Item

Type: {rubric_type}

Content: {rubric_text}

Rubric Type Definitions

- **SUGGEST:** A recommended reasoning step or approach for solving the problem.
- **PITFALL:** A common mistake or error that students might make.
- **BONUS:** An alternative valid approach or insight.
- **ANSWER:** The expected final answer.

Your Task

Judge whether this rubric item is **valid** for the given math problem:

1. Is the content mathematically correct?
2. Is it relevant to this specific problem?
3. Is the type label (SUGGEST/PITFALL/BONUS/ANSWER) appropriate?

Respond with **ONLY** a JSON object (no markdown, no extra text):

{"valid": true or false, "reason": "brief explanation"}

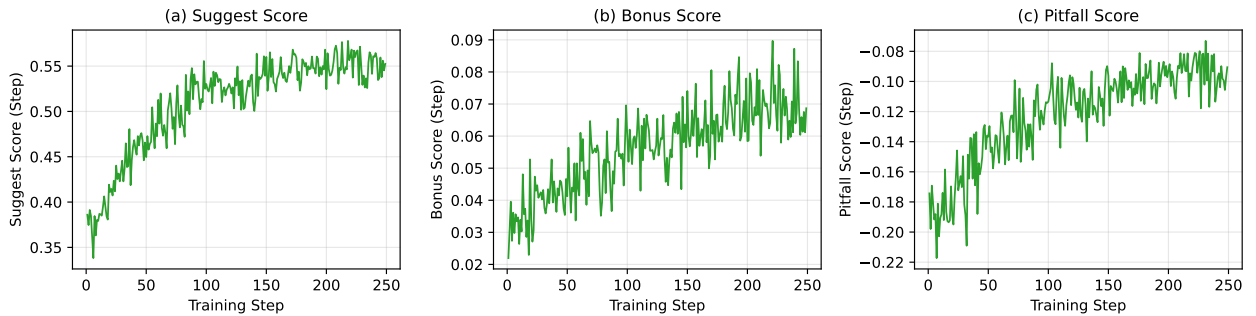


Figure 6 Step-level rubric score dynamics of SRaR during training on Qwen3-8B-Non-Thinking. **(a)** Suggest score steadily increases, indicating the model increasingly follows recommended reasoning steps. **(b)** Bonus score rises as the model discovers alternative solution strategies. **(c)** Pitfall score increases (toward zero), showing the model progressively avoids common errors.

G LLM Judge Agreement Analysis

A core component of SRaR is the LLM judge that evaluates each rubric item against the model’s response and attributes it to a specific reasoning step. To validate the reliability of this judge, we conduct an agreement study on 100 training trajectories sampled uniformly across training steps 20–200 (10 trajectories per checkpoint, totaling 664 rubric items).

Setup

We use GPT-OSS-20B as the LLM judge (identical to the judge used during training) and compare its outputs against independent human annotations by two of the authors on the same 664 rubric items. For each item, both the LLM judge and the human annotators produce a binary satisfied judgment and a step attribution (the step number most closely associated with the rubric item).

Agreement on Rubric Satisfaction

Table 6 reports the agreement between the two independent evaluations.

Table 6 LLM judge agreement on 664 rubric items from 100 training trajectories.

Metric	Value
Satisfied agreement rate	89.3%
Satisfied Cohen’s κ	0.76
Step attribution agreement	80.7%

The two evaluations agree on 89.3% of rubric items with a Cohen’s κ of 0.76, corresponding to substantial agreement on the Landis–Koch scale. For step attribution, when both evaluations assign a specific step number, they agree 80.7% of the time.

Rubric Scores and Answer Correctness

We further analyze the relationship between rubric satisfaction rates and final answer correctness across the 100 trajectories (Table 7).

Table 7 Rubric satisfaction rates for correct vs. incorrect trajectories (100 trajectories, 664 rubric items). Point-biserial correlations are between binary correctness and satisfaction rate.

Rubric Type	Correct	Incorrect	Diff.	r_{pb}	p -value
SUGGEST satisfied	74.2%	24.7%	+49.5 pp	0.62	<0.001
PITFALL triggered	2.5%	24.2%	−21.7 pp	−0.31	0.002
BONUS satisfied	12.5%	0.0%	+12.5 pp	0.28	0.005

All three rubric aspects are significantly correlated with answer correctness ($p < 0.01$). Correct trajectories satisfy 74.2% of SUGGEST items (vs. 24.7% for incorrect ones), trigger only 2.5% of PITFALL items (vs. 24.2%), and satisfy 12.5% of BONUS items (vs. 0%). This confirms that the rubric scores produced by the LLM judge are meaningful indicators of reasoning quality and are well-aligned with outcome correctness.

H Judge Model Robustness

To validate that SRaR is not dependent on a specific LLM judge, we replace the default GPT-OSS-20B judge with an alternative open-source model, Qwen3-30B-A3B-Instruct, while keeping all other training hyperparameters identical. Table 8 reports the results on Qwen3-8B-Non-Thinking.

Table 8 SRaR performance on Qwen3-8B-Non-Thinking with different LLM judges. Both judges produce improvements over the RaR baseline (68.96 avg.), confirming that SRaR’s gains are robust to the choice of judge model.

Judge Model	AIME24	AIME25	AMC23	MATH500	Minerva	Olympiad	Avg.
RaR (no step-level)	57.50	43.23	88.20	92.6	64.71	67.51	68.96
SRaR w/ GPT-OSS-20B	61.15	49.06	92.73	94.4	64.71	73.15	72.53
SRaR w/ Qwen3-30B-A3B	57.81	46.04	91.25	93.20	66.54	70.33	70.86

With Qwen3-30B-A3B-Instruct as the judge, SRaR still achieves a +1.90 average improvement over RaR (70.86 vs. 68.96), demonstrating that the step-wise reward framework is effective with a different judge model. The gap relative to the GPT-OSS-20B judge (−1.67 avg.) is modest, suggesting that judge choice primarily affects the magnitude of improvement rather than the viability of the approach. This is consistent with the expectation that different judges produce varying step attributions, which are partially mitigated by cross-rollout normalization (noise that is uncorrelated across rollouts averages out during normalization).

I Rubric Examples

We present two representative examples from our curated training set to illustrate the structure and quality of the generated rubrics. Each problem is accompanied by a rubric list consisting of SUGGEST, PITFALL, BONUS, and ANSWER items produced by our three-stage pipeline.

Example 1

Problem. Given that nonzero real numbers x and y satisfy $x + \frac{1}{y} = 3$ and $y + \frac{1}{x} = 4$, find the value of $xy + \frac{1}{xy}$.

Ground Truth. 10

Generated Rubric

<SUGGEST> Multiplies the two equations to obtain $\left(x + \frac{1}{y}\right)\left(y + \frac{1}{x}\right) = 12$.
 <SUGGEST> Expands the product correctly to $xy + 2 + \frac{1}{xy}$.
 <SUGGEST> From $xy + 2 + \frac{1}{xy} = 12$, isolates and states $xy + \frac{1}{xy} = 10$.
 <PITFALL> Misexpanding $\left(x + \frac{1}{y}\right)\left(y + \frac{1}{x}\right)$, especially omitting the +2 term from $x \cdot \frac{1}{x}$ and $\frac{1}{y} \cdot y$.
 <BONUS> Computes $xy + \frac{1}{xy}$ directly via the product expansion without solving for x or y individually.
 <ANSWER> Final value reported for $xy + \frac{1}{xy}$ equals 10.

Example 2

Problem. Let S be the set of triples (a, b, c) of non-negative integers such that $a + b + c$ is even. The value of $\sum_{(a,b,c) \in S} \frac{1}{2^a 3^b 5^c}$ can be expressed as $\frac{m}{n}$ where $\gcd(m, n) = 1$. Compute $m + n$.

Ground Truth. 37

Generated Rubric

<SUGGEST> Applies the parity indicator $\frac{1+(-1)^{a+b+c}}{2}$ to rewrite the restricted sum as $\frac{S_0+S_1}{2}$, where S_0 is the unrestricted sum and S_1 includes the factor $(-1)^{a+b+c}$.

<SUGGEST> Factors S_0 into $\sum \frac{1}{2^a} \cdot \sum \frac{1}{3^b} \cdot \sum \frac{1}{5^c}$ and evaluates to $2 \cdot \frac{3}{2} \cdot \frac{5}{4} = \frac{15}{4}$.

<SUGGEST> Factors S_1 into $\sum \frac{(-1)^a}{2^a} \cdot \sum \frac{(-1)^b}{3^b} \cdot \sum \frac{(-1)^c}{5^c}$ and evaluates to $\frac{2}{3} \cdot \frac{3}{4} \cdot \frac{5}{6} = \frac{5}{12}$.

<SUGGEST> Combines via $\frac{S_0+S_1}{2} = \frac{15/4+5/12}{2} = \frac{25}{12}$, giving $m+n=37$.

<PITFALL> Omits the $\frac{1}{2}$ factor in the parity projection, or uses $\frac{S_0-S_1}{2}$ (which sums over odd $a+b+c$) instead of $\frac{S_0+S_1}{2}$.

<BONUS> Uses the compact form $\frac{1}{2} \left(\frac{1}{(1-x)(1-y)(1-z)} + \frac{1}{(1+x)(1+y)(1+z)} \right)$ with $x = \frac{1}{2}, y = \frac{1}{3}, z = \frac{1}{5}$.

<ANSWER> Final reported $m+n$ equals 37.

J Case Studies

We provide representative case studies from AIME 2025 (repeat_id=0) to illustrate the qualitative differences between RaR and SRaR identified in our analysis.

J.1 Process--Outcome Alignment Case Study

We present two problems where RaR produces incorrect answers with step-level errors, while SRaR produces correct answers with all steps judged correct. **Case 1: AIME 2025 Problem 20.** Circle ω_1 with radius 6 centered at point A is internally tangent at point B to circle ω_2 with radius 15... The area of rectangle $EFGH$ is $\frac{m}{n}$... Find $m+n$. (Ground truth: 293.)

RaR Response (Problem 20) — Incorrect, Looping

Step 1: Set up coordinates

We can place the center of ω_2 at the origin... Since the circles are internally tangent at point B ... the center A of ω_1 lies along the line connecting the centers...

[The model enters an extended self-revision loop with 34 revision phrases across 30,922 characters. It repeatedly reconsiders the geometric constraints without converging. The response is cut off mid-computation without producing a \boxed{} answer.]

...So the inequality becomes: $33 - p < p + 15$... 18

[Response truncated — no final answer produced.]

SRaR Response (Problem 20) — Correct, All Steps Correct

Step 1: Understand the configuration and setup coordinates

We are given two circles, ω_1 and ω_2 ... Let's place the center of ω_2 at the origin. Then $A = (9, 0)$, $O = (0, 0)$, $B = (15, 0)$.

Step 2: Find key points

Since BC is a diameter of ω_2 , $C = (-15, 0)$. Since $BC \perp AD$, D lies directly above A ... $D = (9, 12)$.

Step 3: Compute the rectangle and verify equal areas

...area of the rectangle is $288/5$, so $m = 288$, $n = 5$, $m+n = 293$.

Step 4: Final Answer

\boxed{293}

Analysis. RaR's trajectory-level reward fails to prevent the model from entering an extended self-revision loop: it repeatedly reconsiders the geometric constraints without converging, ultimately exhausting the generation budget across 30,922 characters. In contrast, SRaR's step-wise reward allows the model to commit to each step and progress efficiently through the solution in four clean steps with only 6,640 characters. **Case 2: AIME 2025 Problem 24.** Sixteen chairs are arranged in a row. Eight people each select a chair in which

to sit so that no person sits next to two other people. Let N be the number of subsets. . . Find the remainder when N is divided by 1000. (Ground truth: 907.)

RaR Response (Problem 24) — Incorrect, Unstructured Looping

We are given 16 chairs arranged in a row. . . Wait, actually the problem says “no person sits next to two other people.” Hmm, let me parse that carefully.

[The model never produces any ### Step headers. Instead, it generates an unstructured stream of reasoning with 63 self-correction phrases across 33,378 characters. It repeatedly restarts the dynamic programming computation without converging. The response ends mid-computation:]

```
...k=6: dp[7][6] + dp[6][5] + dp[5][4] = 0 + 0 + 1 = 1
k=7: 0
k=8: 0
n=8:
k=0: 1
k=1: 8
```

[Response truncated — no final answer produced.]

SRaR Response (Problem 24) — Correct, All Steps Correct

Step 1: Understand the problem

We have 16 chairs arranged in a row. Eight people each select a chair. . . The condition means no person is adjacent to two other people, i.e., no three consecutive chairs are occupied.

Step 2: Count the valid subsets

. . . our calculation of 2907 should be correct. Thus the answer is 907.

Step 3: Compute the final answer

The remainder when N is divided by 1000 is $\boxed{907}$.

Analysis. Both models initially grapple with interpreting the constraint, but RaR degenerates into an unstructured stream of 63 self-corrections without ever organizing its reasoning into steps, ultimately truncating without an answer. SRaR organizes the solution into three clear steps, correctly interprets the constraint, and arrives at the right answer.

J.2 Looping Case Study

We present AIME 2025 Problem 13 as an illustrative example of the looping behavior analyzed in Section 4.3.2.

Let $ABCDE$ be a convex pentagon with $AB = 14$, $BC = 7$, $CD = 24$, $DE = 13$, $EA = 26$, and $\angle B = \angle E = 60^\circ$. For each point X in the plane, define $f(X) = AX + BX + CX + DX + EX$. The least possible value of $f(X)$ can be expressed as $m + n\sqrt{p}$. . . Find $m + n + p$. (Ground truth: 60.)

RaR Response (Problem 13) — Severe Looping, No Answer

Step 1: Understanding the Problem

The function $f(X)$ represents the sum of distances from a variable point X to each of the five vertices. . . This is the Fermat-Torricelli problem generalized to five points. . .

[The model enters a revision loop with 31 self-correction phrases across 23,874 characters. It repeatedly restarts the coordinate geometry computation without converging. Representative excerpts:]

```
...Wait, if DA is (a, b), then perpendicular vectors are (-b, a) and (b, -a). . .
...Wait, no: Wait, if DA is (a, b). . .
```

[The response ends mid-sentence without producing a $\boxed{}$ answer.]

SRaR Response (Problem 13) — Structured, Produces Answer

Step 1: Understand the problem and identify key elements

We are given a convex pentagon $ABCDE$ with specific side lengths and angles at B and E each equal to 60° . We need to find the minimum of $f(X) = AX + BX + CX + DX + EX \dots$

Step 2: Apply geometric reasoning

...the total minimal value might be $7\sqrt{3} + 13\sqrt{3} = 20\sqrt{3}$ plus other terms...

[The model works through the problem with self-correction (11 phrases), but stays structured and converges to an answer.]

\boxed{58} (incorrect, but a complete structured response)

Analysis. This problem is difficult for both models (neither answers correctly), but the behavioral contrast is stark. RaR produces a response with 31 self-correction phrases, repeatedly restarting its coordinate computation and ultimately failing to produce any answer across 23,874 characters. SRaR, guided by step-wise rewards, maintains a clear two-step structure and converges to a definite (though incorrect) answer in 15,483 characters. This illustrates how step-wise rewards act as a structural prior that prevents degeneration into aimless revision, even on problems the model cannot ultimately solve.

K Data-Size Sensitivity

We analyze how model performance changes with the amount of synthetic data. Instead of varying task types, we vary only the number of synthetic rubrics training samples. Concretely, we train for exactly one epoch on subsets of size 1k, 2k, 4k, 8k, and 16k. All subsets are obtained by random sampling with a fixed seed, so improvements mainly reflect data amount rather than selection artifacts.

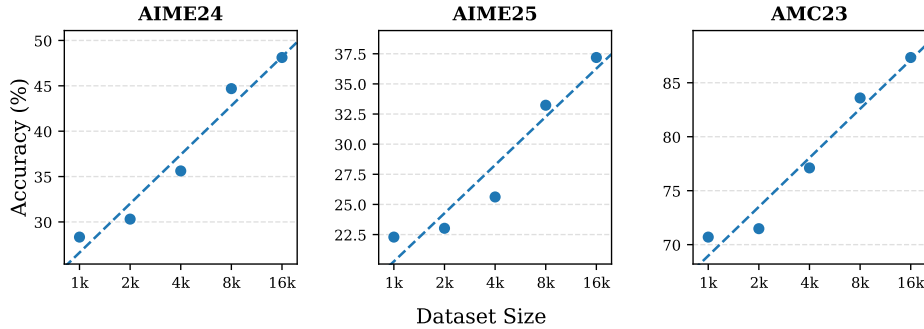


Figure 7 Data-Size Effects of Synthetic Rubrics. We report performance trends on AIME 2024, AIME 2025, and AMC 2023 as the amount of synthetic rubrics training data increases. “1k/2k/4k/8k/16k” are random subsets sampled with a fixed seed. The training recipe matches the main experimental setting. Results show a consistent upward trend across all benchmarks as synthetic data increases.

Figure 7 reports accuracy across three benchmarks as data size scales from 1k to 16k. Several observations stand out. First, performance improves monotonically on every benchmark: even the smallest 1k subset already yields meaningful gains, while each doubling of data continues to push accuracy higher, confirming that our synthetic rubrics pipeline produces consistently useful training signal. Second, the steepest improvement occurs between 4k and 8k samples. For instance, AIME25 jumps from 25.62% to 33.23%, and AMC23 rises from 77.11% to 83.59%, suggesting a critical mass of rubric diversity is reached in this regime. Third, at the full 16k scale, the curves have not yet plateaued. For example, AIME24 reaches 48.12%, AIME25 reaches 37.19%, and AMC23 reaches 87.34%, indicating that further scaling the synthetic data is likely to yield additional gains. Finally, the trend is consistent across benchmarks of different difficulty levels, from the easier AMC23 (70.70%→87.34%) to the harder AIME25 (22.29%→37.19%), demonstrating that the benefit of more synthetic rubrics is not confined to a narrow difficulty band.

L Rubric Aspect Ablation

We ablate the rubric aspects to disentangle the contribution of step-wise credit assignment from the rubric design. Table 9 compares RaR and SRaR under two rubric configurations on Qwen3-8B-Non-Thinking: Suggest-only (removing PITFALL and BONUS) and the full multi-aspect set.

Table 9 Rubric aspect ablation on Qwen3-8B-Non-Thinking. Δ denotes SRaR improvement over RaR within the same rubric configuration. Best results per configuration are **bolded**.

Method	AIME24	AIME25	AMC23	MATH500	Minerva	Olympiad	Avg.
<i>SUGGEST-only rubrics</i>							
RaR	53.23	41.98	91.64	93.2	61.40	69.73	68.53
SRaR	58.65	46.98	91.56	93.6	66.18	71.36	71.39
Δ	+5.42	+5.00	-0.08	+0.40	+4.78	+1.63	+2.86
<i>Full rubrics (SUGGEST + PITFALL + BONUS)</i>							
RaR	57.50	43.23	88.20	92.6	64.71	67.51	68.96
SRaR	61.15	49.06	92.73	94.4	64.71	73.15	72.53
Δ	+3.65	+5.83	+4.53	+1.80	+0.00	+5.64	+3.57

Two conclusions emerge. First, SRaR outperforms RaR under both rubric configurations (+2.86 with Suggest-only, +3.57 with full rubrics), confirming that the primary source of improvement is step-wise credit assignment rather than the specific rubric aspects. Second, upgrading from Suggest-only to full rubrics improves SRaR from 71.39 to 72.53 (+1.14), indicating that PITFALL and BONUS provide complementary supervisory signal that further benefits step-wise training.

M Method Design Ablation

We ablate three key design choices of SRaR on Qwen3-8B-Non-Thinking to isolate their individual contributions. Table 10 reports results across all six benchmarks.

Table 10 Component ablation on Qwen3-8B-Non-Thinking. Each variant removes one component from the full SRaR system. Best results are **bolded**.

Method	AIME24	AIME25	AMC23	MATH500	Minerva	Olympiad	Avg.
Base (Qwen3-8B, w/ step format)	26.77	22.40	71.88	85.0	55.51	56.53	53.02
DAPO (w/o step format)	55.73	41.25	87.66	92.0	61.76	68.69	67.85
SRaR (w/o step norm.)	45.10	28.96	73.52	87.2	49.26	56.53	56.76
SRaR (w/o decoupled adv.)	62.60	46.67	93.28	94.6	64.71	70.47	72.06
SRaR (full)	61.15	49.06	92.73	94.4	64.71	73.15	72.53

Step-structured format prompt. Comparing DAPO trained without step format constraints (67.85 avg) against the full DAPO baseline in Table 1 (67.26 avg), the two are virtually identical, indicating that requiring the model to organize its chain-of-thought into explicit **### Step** blocks is a benign structural constraint that introduces no measurable performance penalty. This makes it a practical prerequisite for SRaR, which relies on step boundaries to attribute rubric items to individual reasoning steps. **Cross-rollout step normalization.** Removing step normalization causes severe training collapse (56.76 avg), performing even worse than the base model on several benchmarks. Without normalization, rubric items that are uniformly satisfied across all rollouts still produce large positive advantages. The model exploits this by collapsing all reasoning into a single **### Step 1**, maximizing rubric reward without distributing reasoning across multiple steps. We provide a representative example in Appendix N. Cross-rollout normalization prevents this by zeroing

out the advantage for rubric items whose scores do not vary across rollouts, ensuring that only genuinely discriminative rubric items drive learning. **Decoupled advantage estimation.** Instead of maintaining separate outcome and rubric advantage streams, this variant computes a single RaR-style scalar reward (combining outcome correctness and rubric satisfaction) and then allocates it to the tokens of steps that received rubric scores. This approach yields competitive performance (72.06 avg) but falls short of the full SRaR (72.53 avg). The gap is most pronounced on harder benchmarks (AIME 2025: 46.67 vs. 49.06), suggesting that merging outcome and rubric signals into one scalar before allocation loses the benefit of keeping the outcome-driven baseline stable.

N Case Study: Training Collapse without Step Normalization

Below is a representative response from the SRaR (w/o step normalization) model after training collapse. The model places its entire chain-of-thought (spanning over 4,000 tokens of deliberation) inside a single `### Step 1`, never producing any subsequent step. This behavior emerges because, without cross-rollout normalization, rubric items that are consistently satisfied across all rollouts (e.g., “identify the modular invariant”) still carry large positive advantages. By concentrating all reasoning in one step, the model ensures that this single step collects all rubric credit, removing any incentive to distribute reasoning across multiple steps.

SRaR (w/o step norm.) – Single-Step Collapse

```
### Step 1: Understand the problem and analyze the operations
We are given a set of numbers from 1 to 2004 written on a desk. A move consists of selecting some numbers,
calculating the remainder of their sum modulo 11, writing this remainder on the desk, and deleting the selected
numbers. After several such moves, only two numbers remain on the desk, one of which is 1000. We are to
find the other number.
First, let's understand the invariant... Each move replaces some numbers with their sum modulo 11. So, if
we consider the numbers on the desk as elements of the set, the key is that the sum of all numbers on the
desk modulo 11 is preserved...
[... 4,000+ tokens of reasoning, all within Step 1, including repeated self-verification, alternative approaches,
and redundant restatements...]
Therefore, the answer is 4.
### Final Answer
\boxed{4}
```

Analysis. The response contains a single reasoning step that spans the entire solution. While the final answer happens to be correct for this particular problem, the degenerate structure, including no step decomposition and excessive self-verification loops within one monolithic block, is a direct consequence of the un-normalized rubric reward. In contrast, the full SRaR model with cross-rollout normalization produces well-structured multi-step solutions, because normalization zeros out the advantage for rubric items that do not vary across rollouts, leaving only genuinely discriminative items to shape the policy gradient.

O Theoretical Properties of SRaR

We provide formal analyses of the key properties of SRaR. Throughout this section, we consider a single prompt x with n rollouts $\{y_1, \dots, y_n\}$ sampled from the policy $\pi_\theta(\cdot|x)$. For clarity, we first state the zero-sum properties under the idealized setting $\epsilon = 0$ and then note that the practical implementation with $\epsilon > 0$ introduces negligible deviation (Remark O.4).

O.1 Zero-Sum Properties

Proposition O.1 (Zero-Sum of GRPO Outcome Advantage). *Let $r_{base,i}$ denote the base reward (accuracy + format) of rollout i , and define the GRPO outcome advantage as*

$$A_{base,i} = \frac{r_{base,i} - \mu_{base}}{\sigma_{base}}, \quad \mu_{base} = \frac{1}{n} \sum_{i=1}^n r_{base,i}, \quad \sigma_{base}^2 = \frac{1}{n} \sum_{i=1}^n (r_{base,i} - \mu_{base})^2. \quad (7)$$

Then $\sum_{i=1}^n A_{base,i} = 0$.

Proof. By linearity:

$$\sum_{i=1}^n A_{base,i} = \frac{1}{\sigma_{base}} \sum_{i=1}^n (r_{base,i} - \mu_{base}) = \frac{1}{\sigma_{base}} \left(\sum_{i=1}^n r_{base,i} - n\mu_{base} \right) = \frac{1}{\sigma_{base}} (n\mu_{base} - n\mu_{base}) = 0. \quad (8)$$

Proposition O.2 (Zero-Sum of Step-wise Normalized Rubric Signals). *For a given step number k , let $\mathcal{G}_k \subseteq \{1, \dots, n\}$ be the set of rollout indices whose judge evaluation contains at least one rubric item attributed to step k , and let $d_{k,i}$ be the raw aggregate rubric delta for step k in rollout $i \in \mathcal{G}_k$. Define the step-wise normalized signal as*

$$\bar{d}_{k,i} = \frac{d_{k,i} - \mu_k}{\sigma_k}, \quad \mu_k = \frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} d_{k,i}, \quad \sigma_k^2 = \frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} (d_{k,i} - \mu_k)^2. \quad (9)$$

Then $\sum_{i \in \mathcal{G}_k} \bar{d}_{k,i} = 0$.

Proof.

$$\sum_{i \in \mathcal{G}_k} \bar{d}_{k,i} = \frac{1}{\sigma_k} \sum_{i \in \mathcal{G}_k} (d_{k,i} - \mu_k) = \frac{1}{\sigma_k} \left(\sum_{i \in \mathcal{G}_k} d_{k,i} - |\mathcal{G}_k| \cdot \mu_k \right) = 0. \quad (10)$$

Corollary O.3 (Zero-Sum Decomposition of SRaR Advantage). *The SRaR advantage $\hat{A}_i^{(t)} = A_{base,i} + \hat{r}_i^{(t)}$ decomposes into two independently zero-sum components across rollouts:*

1. *The outcome component: $\sum_{i=1}^n A_{base,i} = 0$ (Proposition O.1);*
2. *The rubric component: for each step k , $\sum_{i \in \mathcal{G}_k} \bar{d}_{k,i} = 0$ (Proposition O.2).*

Proof. Follows directly from Propositions O.1 and O.2. The outcome advantage is a scalar broadcast to all tokens, so it sums to zero over rollouts at every token position. The rubric reward at any token t belonging to step k in rollout i equals $\bar{d}_{k,i}$, which sums to zero over $\mathcal{G}_k$ by Proposition O.2. The two components are computed from disjoint information (base outcome vs. rubric evaluations) and normalized independently, hence their zero-sum properties hold simultaneously. $\square$

Remark O.4 (Effect of ϵ). In practice, a small constant $\epsilon > 0$ is added to the denominator for numerical stability. This introduces a bounded deviation:

$$\left| \sum_{i \in \mathcal{G}_k} \bar{d}_{k,i}^{(\epsilon)} \right| = \left| \frac{\sum_i (d_{k,i} - \mu_k)}{\sigma_k + \epsilon} \right| = 0, \quad (11)$$

since the numerator $\sum_i (d_{k,i} - \mu_k) = 0$ regardless of the denominator. Therefore, the zero-sum property holds exactly even with $\epsilon > 0$.

O.2 Bounded Reward Analysis

Proposition O.5 (Bounded Per-Sample Rubric Reward). *For any single rollout i , the total raw rubric reward $\Delta_i = \sum_k d_{k,i}$ is bounded:*

$$R_{PIT} \leq \Delta_i \leq R_{SUG} + R_{BON} \quad (12)$$

where $R_{SUG} > 0$, $R_{PIT} < 0$, and $R_{BON} > 0$ are the pre-defined total reward budgets for each rubric type.

Proof. Consider each rubric type separately. Let N_τ denote the number of items of type τ in the current sample.

SUGGEST items. Each satisfied item contributes $+R_{SUG}/N_{SUG}$; unsatisfied items contribute 0. Since at most N_{SUG} items can be satisfied, the total SUGGEST contribution lies in $[0, R_{SUG}]$.

PITFALL items. Each satisfied (i.e., the model fell into the pitfall) item contributes $-|R_{\text{PIT}}|/N_{\text{PIT}}$; unsatisfied items contribute 0. The total PITFALL contribution lies in $[R_{\text{PIT}}, 0]$.

BONUS items. By the same argument, the total BONUS contribution lies in $[0, R_{\text{BON}}]$.

Summing over all types: $\Delta_i \in [R_{\text{PIT}}, R_{\text{SUG}} + R_{\text{BON}}]$. $\square$

Proposition O.6 (Unit Variance of Normalized Step Signals). *For a given step k with $|\mathcal{G}_k| \geq 2$ and $\sigma_k > 0$, the normalized deltas satisfy:*

$$\frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} \bar{d}_{k,i}^2 = 1. \quad (13)$$

Proof.

$$\frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} \bar{d}_{k,i}^2 = \frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} \frac{(d_{k,i} - \mu_k)^2}{\sigma_k^2} = \frac{1}{\sigma_k^2} \cdot \frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} (d_{k,i} - \mu_k)^2 = \frac{\sigma_k^2}{\sigma_k^2} = 1. \quad (14) \quad \square$$

O.3 Reduction to Standard GRPO

Proposition O.7 (Equivalence to GRPO When Rubrics Are Empty). *When the rubric list $\mathcal{R} = \emptyset$ for all training samples, the SRaR advantage estimator reduces to standard GRPO.*

Proof. When $\mathcal{R} = \emptyset$, no LLM judge call is made and no rubric deltas are produced. Thus $d_{k,i} = 0$ for all k, i , and consequently $\bar{d}_{k,i} = 0$. The token-level reward tensor contains only the base reward at the last token: $\mathbf{r}_i[t] = r_{\text{base},i} \cdot \mathbf{1}[t = T_i]$. The decoupled advantage becomes:

$$\hat{A}_i^{(t)} = A_{\text{base},i} + 0 = \frac{r_{\text{base},i} - \mu_{\text{base}}}{\sigma_{\text{base}} + \epsilon}, \quad (15)$$

which is a scalar broadcast to all tokens, identical to the standard GRPO outcome advantage. $\square$

O.4 Policy Gradient Decomposition

Proposition O.8 (Decomposition of the SRaR Policy Gradient). *Let K_i denote the number of steps in rollout i , and let $\mathcal{T}_{k,i}$ denote the set of token indices belonging to step k in rollout i . Under the SRaR advantage, the policy gradient estimate decomposes as:*

$$\hat{g}_{\text{SRaR}} = \hat{g}_{\text{GRPO}} + \hat{g}_{\text{rubric}} \quad (16)$$

where

$$\hat{g}_{\text{GRPO}} = \frac{1}{n} \sum_{i=1}^n A_{\text{base},i} \sum_{t=1}^{T_i} \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i), \quad (17)$$

$$\hat{g}_{\text{rubric}} = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^{K_i} \bar{d}_{k,i} \sum_{t \in \mathcal{T}_{k,i}} \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i). \quad (18)$$

Proof. The SRaR policy gradient estimate is:

$$\hat{g}_{\text{SRaR}} = \frac{1}{n} \sum_{i=1}^n \sum_{t=1}^{T_i} \hat{A}_i^{(t)} \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i). \quad (19)$$

Substituting $\hat{A}_i^{(t)} = A_{\text{base},i} + \tilde{r}_i^{(t)}$ and using linearity of summation:

$$\hat{g}_{\text{SRaR}} = \underbrace{\frac{1}{n} \sum_{i=1}^n A_{\text{base},i} \sum_{t=1}^{T_i} \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i)}_{\hat{g}_{\text{GRPO}}} + \underbrace{\frac{1}{n} \sum_{i=1}^n \sum_{t=1}^{T_i} \tilde{r}_i^{(t)} \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i)}_{\hat{g}_{\text{rubric}}}. \quad (20)$$

For the rubric term, since $\tilde{r}_i^{(t)} = \bar{d}_{k,i}$ for all $t \in \mathcal{T}_{k,i}$ (full-span placement) and $\tilde{r}_i^{(t)} = 0$ for tokens not belonging to any step with rubric evaluations, we can rewrite:

$$\hat{g}_{\text{rubric}} = \frac{1}{n} \sum_{i=1}^n \sum_{k=1}^{K_i} \bar{d}_{k,i} \sum_{t \in \mathcal{T}_{k,i}} \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i). \quad (21)$$

Remark O.9 (Interpretation). Equation (17) is exactly the standard GRPO gradient, which uniformly up-weights or downweights the log-probability of the entire response based on outcome correctness. Equation (18) provides a step-selective correction: for each step k , the log-probabilities of tokens within that step are scaled by $\bar{d}_{k,i}$, which is positive when the step is relatively better than the same step in other rollouts, and negative otherwise. This allows the gradient to selectively reinforce or suppress individual reasoning steps within an otherwise uniformly scored response.

O.5 Variance Analysis

Proposition O.10 (Scale Alignment Between Outcome and Rubric Signals). *After normalization, both the outcome advantage and the step-wise rubric signals have controlled second moments:*

$$\frac{1}{n} \sum_{i=1}^n A_{\text{base},i}^2 = 1, \quad (22)$$

$$\frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} \bar{d}_{k,i}^2 = 1, \quad \forall k. \quad (23)$$

Proof. For the outcome advantage (Eq. 22), using population variance $\sigma_{\text{base}}^2 = \frac{1}{n} \sum_i (r_{\text{base},i} - \mu_{\text{base}})^2$:

$$\frac{1}{n} \sum_{i=1}^n A_{\text{base},i}^2 = \frac{1}{n \sigma_{\text{base}}^2} \sum_{i=1}^n (r_{\text{base},i} - \mu_{\text{base}})^2 = \frac{n \sigma_{\text{base}}^2}{n \sigma_{\text{base}}^2} = 1. \quad (24)$$

For the step-wise rubric signal (Eq. 23), using population variance $\sigma_k^2 = \frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} (d_{k,i} - \mu_k)^2$:

$$\frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} \bar{d}_{k,i}^2 = \frac{1}{|\mathcal{G}_k| \sigma_k^2} \sum_{i \in \mathcal{G}_k} (d_{k,i} - \mu_k)^2 = \frac{|\mathcal{G}_k| \sigma_k^2}{|\mathcal{G}_k| \sigma_k^2} = 1. \quad (25)$$

Both quantities equal 1, confirming that the two signals operate on exactly the same scale. $\square$

Remark O.11 (Variance Reduction via Step-wise Normalization). Without step-wise normalization (e.g., reward pattern 1 or 2), the raw rubric deltas $d_{k,i}$ may have highly variable magnitudes depending on the number and type of rubric items. In the worst case, the rubric signal can dominate or be negligible relative to the outcome advantage, destabilizing training. Step-wise normalization guarantees unit population variance per step (Proposition O.6), preventing any single step from contributing disproportionately to the gradient. Combined with the outcome advantage also having unit population variance (Proposition O.10), the two components of the SRaR advantage remain on the same scale throughout training, regardless of the raw rubric reward magnitudes.

O.6 Unbiasedness of the Policy Gradient Estimator

Proposition O.12 (SRaR Preserves Unbiasedness of the REINFORCE Estimator). *Define the SRaR reward for rollout i at token t as $R_i^{(t)} = r_{\text{base},i} + \tilde{r}_i^{(t)}$, where $\tilde{r}_i^{(t)}$ is the rubric reward at token t . Then the SRaR policy gradient estimator*

$$\hat{g}_{\text{SRaR}} = \frac{1}{n} \sum_{i=1}^n \sum_{t=1}^{T_i} (R_i^{(t)} - b_i^{(t)}) \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \quad (26)$$

is an unbiased estimator of $\nabla_{\theta} J_{\text{SRaR}}(\theta)$ for any baseline $b_i^{(t)}$ that is independent of the action a_t^i , where $J_{\text{SRaR}}(\theta) = \mathbb{E}_{y \sim \pi_{\theta}}[R(y)]$ is the expected SRaR reward.

Proof. This follows from the standard REINFORCE theorem [17]. The rubric reward $\hat{r}_i^{(t)}$ is a function of the complete response y_i (evaluated by the judge after generation), thus it is a valid return signal. For any baseline $b_i^{(t)}$ that does not depend on a_t^i :

$$\mathbb{E}_{a_t^i \sim \pi_\theta} \left[b_i^{(t)} \nabla_\theta \log \pi_\theta(a_t^i | s_t^i) \right] = b_i^{(t)} \nabla_\theta \sum_a \pi_\theta(a | s_t^i) = b_i^{(t)} \nabla_\theta 1 = 0. \quad (27)$$

Therefore subtracting the baseline does not introduce bias.

In the SRaR decoupled advantage, the group-level statistics μ_{base} , σ_{base} , μ_k , and σ_k serve as components of the baseline. In the leave-one-out interpretation, these statistics computed from the other $n - 1$ rollouts are independent of a_t^i for any given rollout i , preserving unbiasedness. When computed including rollout i (as in the standard GRPO implementation), the resulting bias is $O(1/n)$ and vanishes as the group size grows, consistent with the analysis of GRPO $\square$

P Dataset Statistics

Our training set consists of 16,560 math problems, each annotated with a structured rubric composed of four tag types: SUGGEST (key solution steps), PITFALL (common errors), BONUS (alternative approaches or insights), and ANSWER (final answer verification). We provide a detailed statistical analysis below.

P.1 Overall Summary

Table 11 Overall dataset statistics.

Statistic	Value
Total samples	16,560
Mean rubric items per sample	7.53
Median rubric items per sample	8
Mean words per rubric item	18.5

P.2 Rubric Item Counts by Type

Table 12 Statistics of rubric item counts by tag type.

Tag Type	Mean	Std	Median	Min	Max	Avg. Ratio
SUGGEST	4.59	0.86	5	2	8	60.9%
PITFALL	1.26	0.51	1	0	3	16.7%
BONUS	0.67	0.47	1	0	2	8.9%
ANSWER	1.00	0.00	1	1	1	13.3%
Total	7.53	0.79	8	4	12	100%

P.3 Distribution of Total Rubric Item Count

Figure 8 shows the distribution of total rubric item count per sample.

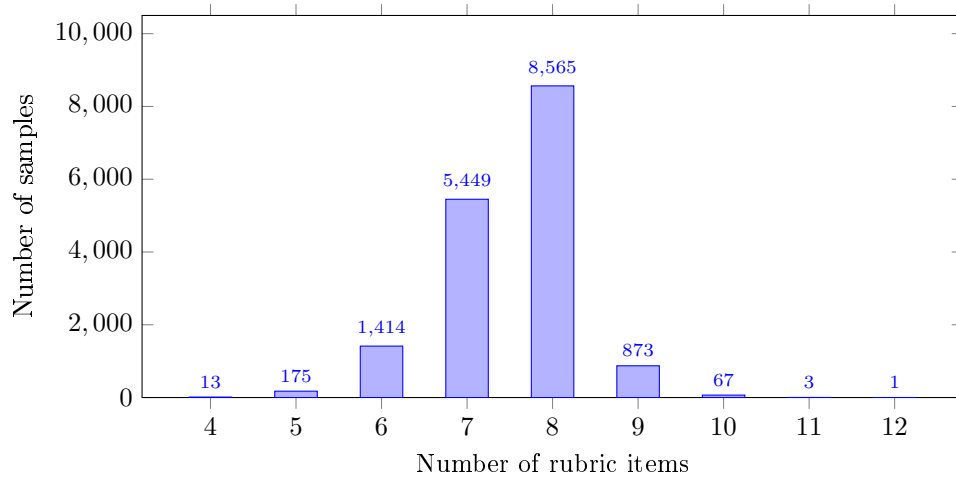


Figure 8 Distribution of total rubric item count per sample. The majority of samples (84.6%) contain 7–8 rubric items.

P.4 Distribution of Suggest Count

Figure 9 shows the distribution of SUGGEST items per sample.

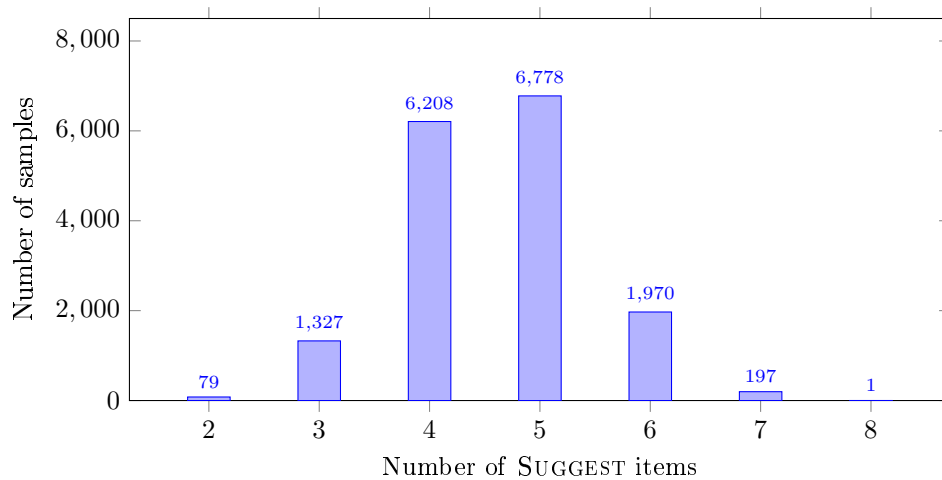


Figure 9 Distribution of SUGGEST items per sample. Most samples contain 4–5 solution steps.

P.5 Distribution of Pitfall Count

Figure 10 shows the distribution of PITFALL items per sample.

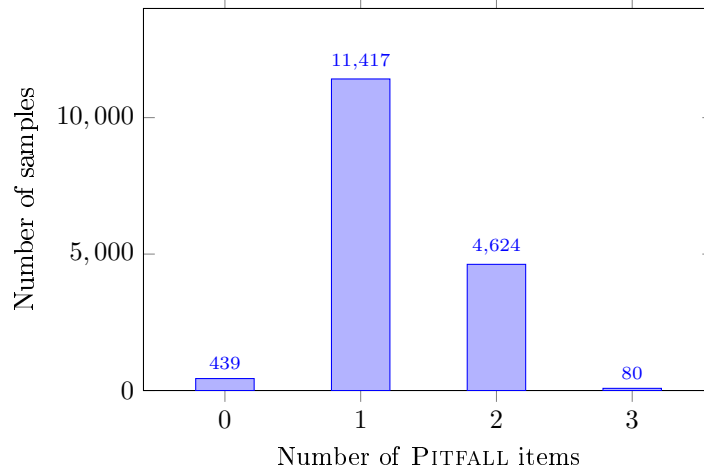


Figure 10 Distribution of PITFALL items per sample. 97.3% of samples have 1–2 pitfall warnings.

P.6 Distribution of Bonus Count

Figure 11 shows the distribution of BONUS items per sample.

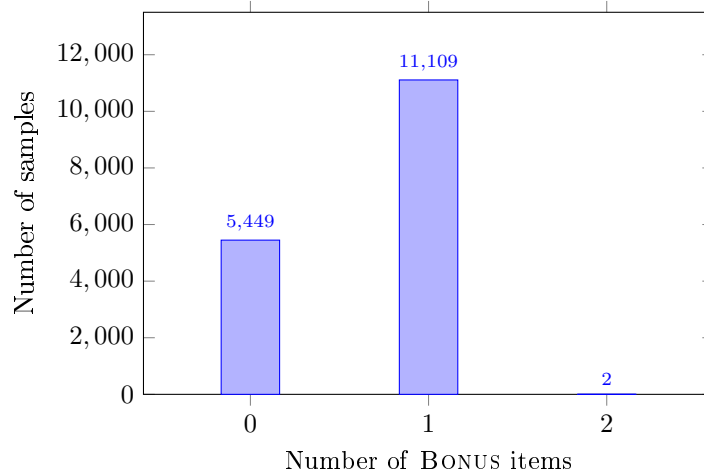


Figure 11 Distribution of BONUS items per sample. 67.1% of samples include one bonus insight.

P.7 Co-occurrence of Pitfall and Bonus

Table 13 Co-occurrence matrix of PITFALL and BONUS item counts (# of samples).

	Bonus=0	Bonus=1	Bonus=2	Total
Pitfall=0	369	70	0	439
Pitfall=1	3,416	8,000	1	11,417
Pitfall=2	1,617	3,006	1	4,624
Pitfall=3	47	33	0	80
Total	5,449	11,109	2	16,560